Chapter 6. Neural Networks and In-Context Learning for Tabular Data

Predictive Modeling, Lecture 5 (MLP, embeddings, backpropagation, Adam, regularization, TabPFN, TabICL)

Contents

1 When Tree Boosting Is Not Enough **2**

2 The MLP as a Composite Nonlinear Function **2**

2.1 One hidden layer 2

3 Neural-Network Inputs Are Numbers **3**

4 Categorical Embeddings Are Learned From Prediction Error **3**

5 Training an MLP: Forward, Backward, Update **4**

5.1 Minibatches and Adam 4

6 The Core Difficulty of Tabular MLPs: Overfitting **5**

7 Trees and MLPs Have Different Inductive Biases **5**

8 Another Route: In-Context Learning **6**

8.1 Refitting versus conditioning 6

8.2 TabPFN pre-trains the learning algorithm itself 6

8.3 TabICL summarizes rows first, then learns in context 6

9 Summary **7**

1 When Tree Boosting Is Not Enough

Limitation of tree boosting		What neural networks offer
finds splits on given features but does not learn the representation of features	$\longrightarrow$	learns representations needed for prediction through **embeddings** and **hidden layers**
smooth relations may need many splits and trees	$\longrightarrow$	combines several features continuously to represent complex relations
text, image, time series must first be summarized into tabular features	$\longrightarrow$	different input types can be learned in one model
usually re-learns a model from scratch for each new data set	$\longrightarrow$	pre-training and in-context learning reuse information from other problems

$$\underbrace{\text{fixed features} \longrightarrow \text{split learning}}_{\text{tree boosting}} \qquad \underbrace{\text{representation learning} + \text{prediction learning}}_{\text{neural network}}$$

2 The MLP as a Composite Nonlinear Function

2.1 One hidden layer

$$\begin{aligned} z^{(1)} &= W^{(1)}x + b^{(1)} && \text{weighted sums of the raw features,} \\ a^{(1)} &= \mathrm{ReLU}(z^{(1)}) && \text{hidden representation after activation,} \\ \hat{y} &= g(W^{(2)}a^{(1)} + b^{(2)}) && \text{final prediction from the hidden representation.} \end{aligned}$$

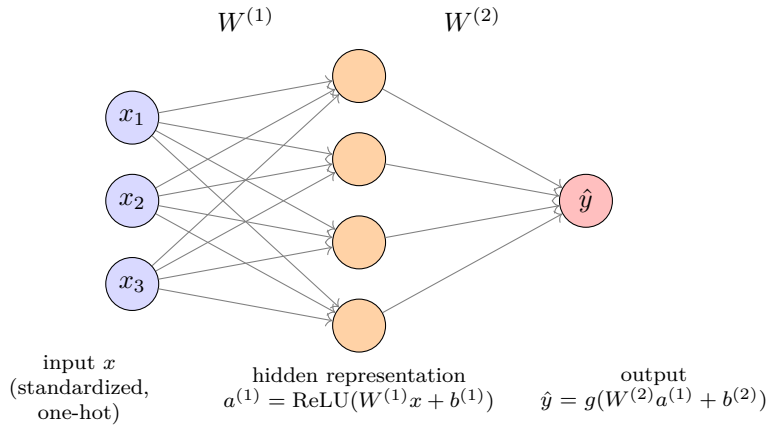


Figure 1: A one-hidden-layer MLP. Each hidden unit forms a weighted sum of all inputs and applies ReLU; the output layer combines the hidden units and applies g (identity, sigmoid or softmax). Backpropagation sends the error signal $\hat{y} - y$ back along the same edges.

Hidden layer: builds linear combinations of the features in many directions; the activation makes piecewise and interaction structure representable; more layers and width enlarge the function space but also the number of parameters. A one-hidden-layer ReLU network is a continuous piecewise-linear function of x, whose pieces are cut by the hyperplanes $\{w_k^{(1)\top}x + b_k^{(1)} = 0\}$; with enough units it can approximate any continuous function on a compact set (universal approximation).

Output layer: the same hidden representation uses a different loss and output layer depending on the target:

Target	Output g	Loss	Gradient w.r.t. the output score
regression	identity $g(u) = u$	$\frac{1}{2}(y - \hat{y})^2$	$\hat{y} - y$
binary	sigmoid	log loss	$\hat{p} - y$
multiclass	softmax $p_k = e^{u_k} / \sum_l e^{u_l}$	cross-entropy $-\sum_k y_k \log p_k$	$p - y$

Proof of the softmax gradient. $\log p_k = u_k - \log \sum_l e^{u_l}$, so $\partial \log p_k / \partial u_m = \mathbb{I}(k = m) - p_m$. Then $\partial(-\sum_k y_k \log p_k)/\partial u_m = -y_m + p_m \sum_k y_k = p_m - y_m$ since $\sum_k y_k = 1$. ■

These "canonical" pairs give the same simple error signal $\hat{y} - y$ at the output, which is then propagated backwards.

3 Neural-Network Inputs Are Numbers

Suppose the raw record of one customer is:

Raw data	Conversion to numbers	Input
age 52	standardize with mean and SD (training data)	0.8
occupation: teacher	one-hot vector	$(0, 1, 0)$
income: missing	median imputation and missing indicator	$(40, 1)$

So the network actually receives $(0.8,\ 0,\ 1,\ 0,\ 40,\ 1)$. Unlike trees, MLPs are sensitive to scale: unscaled inputs (here 40) dominate the initial weighted sums and slow gradient descent, so continuous features are usually standardized (with training statistics only).

4 Categorical Embeddings Are Learned From Prediction Error

With K occupations, keep a learnable matrix $E \in \mathbb{R}^{K \times d}$ with $d \ll K$. Each occupation becomes a row index (teacher = 1, professor = 2, developer = 3, ...); a teacher input fetches row 1:

$$e_{\text{teacher}} = E[1, :] = (0.7, -0.2, 0.4), \qquad o_{\text{teacher}}^{\top} E = E[1, :],$$

i.e. the lookup is the same computation as a one-hot vector times a weight matrix.

Algorithm: how an embedding is learned

1. Initialize the entries of E with small random numbers.
2. Use the row of the observed category as network input.
3. Compute the error between truth and prediction.
4. Update, by backpropagation, only the row that was used:

$$E[1, :] \leftarrow E[1, :] - \eta \frac{\partial \ell}{\partial E[1, :]}.$$

Theorem: the gradient touches only the used row

If $e = o_k^{\top} E$ enters the network, then $\dfrac{\partial \ell}{\partial E} = o_k \left(\dfrac{\partial \ell}{\partial e}\right)^{\top}$, a $K \times d$ matrix whose only nonzero row is row k.

Proof. $e_m = \sum_l o_{k,l} E_{lm} = E_{km}$, so $\partial e_m / \partial E_{lm'} = \mathbb{I}(l = k)\mathbb{I}(m = m')$; apply the chain rule. ■

As teacher cases keep arriving, $E[1,:]$ keeps moving in a direction useful for prediction; categories with similar effects end up with nearby rows. The embedding matrix is not given in advance: it is learned together with the other MLP weights from the current training data. Rare categories receive few updates and remain noisy, which is one reason small tabular data favor trees.

5 Training an MLP: Forward, Backward, Update

Algorithm: minibatch training

Initialize the weights θ. For each epoch and each minibatch B_t:

1. **Forward**: compute predictions and the average loss $\widehat{R}_{B_t}(\theta) = \frac{1}{|B_t|}\sum_{i\in B_t} L\{y_i, f_\theta(x_i)\}$.
2. **Backward**: compute the gradient $g_t = \nabla_\theta \widehat{R}_{B_t}(\theta)$.
3. **Update**: move the weights in the opposite direction, $\theta \leftarrow \theta - \eta g_t$.

At the end of each epoch record the validation loss; at the end, restore the weights with the lowest validation loss.

The forward pass computes $\hat{y}_i = f_\theta(x_i)$ with the current weights and compares it with the truth, e.g. $L_i = \frac{1}{2}(y_i - \hat{y}_i)^2$ in regression. The backward pass uses the chain rule to compute the effect of each weight on the loss, $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial w}$; the sign of the gradient gives the direction of correction and its size the strength of the influence.

Derivation: backpropagation for one hidden layer

For $z^{(1)} = W^{(1)}x + b^{(1)}$, $a^{(1)} = \mathrm{ReLU}(z^{(1)})$, $\hat{y} = W^{(2)}a^{(1)} + b^{(2)}$, $L = \frac{1}{2}\|y - \hat{y}\|^2$:

$$\delta^{(2)} = \hat{y} - y, \quad \frac{\partial L}{\partial W^{(2)}} = \delta^{(2)} a^{(1)\top}, \quad \frac{\partial L}{\partial b^{(2)}} = \delta^{(2)},$$

$$\delta^{(1)} = \left(W^{(2)\top}\delta^{(2)}\right) \odot \mathbb{I}\{z^{(1)} > 0\}, \quad \frac{\partial L}{\partial W^{(1)}} = \delta^{(1)} x^\top, \quad \frac{\partial L}{\partial b^{(1)}} = \delta^{(1)}.$$

Each layer reuses the error signal δ of the layer above, so all gradients cost about as much as one forward pass. Hidden units with $z \le 0$ pass no gradient ("dead" ReLU units).

5.1 Minibatches and Adam

A minibatch gradient is an unbiased estimate of the full-data gradient, $\mathbb{E}[g_t] = \nabla\widehat{R}(\theta)$ when batches are drawn uniformly; its noise helps escape poor regions and costs $O(|B_t|)$ per step, well suited to GPU matrix operations. Adam adapts the step size per parameter:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2,$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \ \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \quad \theta \leftarrow \theta - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon},$$

(typically $\beta_1 = 0.9$, $\beta_2 = 0.999$). The bias corrections compensate for $m_0 = v_0 = 0$: if g_t has a constant mean μ, then $\mathbb{E}m_t = (1-\beta_1^t)\mu$. The ratio $\hat{m}/\sqrt{\hat{v}}$ is roughly scale-free, so parameters with consistently small gradients still move. Minibatch, Adam: *optimization* devices.

6 The Core Difficulty of Tabular MLPs: Overfitting

An MLP has many parameters and is flexible, so it can learn even the noise of the training data:

$$\text{training loss keeps decreasing} \;+\; \text{validation loss rises again} \;\Longrightarrow\; \text{overfitting.}$$

Device	Role	Signal to check
small model	limit depth and width, fewer parameters	does even the training loss stay high (underfitting)?
weight decay	penalize large weights	does the train–validation gap shrink?
dropout	randomly switch off some hidden units during training	does validation improve, or does it overfit?
early stopping	restore the weights of the epoch with minimal validation loss	from when does validation loss rise again?

Theorem: weight decay and dropout as regularizers

(i) Adding $\frac{\lambda}{2}\|\theta\|^2$ to the loss changes the gradient step to $\theta \leftarrow (1-\eta\lambda)\theta - \eta g_t$: every step multiplicatively shrinks the weights (Ridge, Ch. 3).
(ii) Inverted dropout replaces a unit a by $\tilde{a} = a\,m/(1-q)$ with $m \sim \text{Bernoulli}(1-q)$, so $\mathbb{E}\tilde{a} = a$. For a linear model $\hat{y} = \beta^\top \tilde{x}$ with dropout on the inputs,

$$\mathbb{E}_m\big[(y - \beta^\top \tilde{x})^2\big] = (y - \beta^\top x)^2 + \frac{q}{1-q}\sum_j \beta_j^2 x_j^2,$$

i.e. dropout behaves like a data-dependent Ridge penalty.

Proof. (i) $\nabla(\widehat{R} + \frac{\lambda}{2}\|\theta\|^2) = g_t + \lambda\theta$.
(ii) $\mathbb{E}[\beta^\top \tilde{x}] = \beta^\top x$ and the m_j are independent with $\text{Var}\{m_j/(1-q)\} = q/(1-q)$, so $\mathbb{E}(y - \beta^\top \tilde{x})^2 = (y - \beta^\top x)^2 + \text{Var}(\beta^\top \tilde{x})$, and $\text{Var}(\beta^\top \tilde{x}) = \sum_j \beta_j^2 x_j^2\, q/(1-q)$. ■

Weight decay, dropout, early stopping: *generalization* devices. Early stopping on a validation fold is also roughly equivalent to L_2 regularization for gradient descent on quadratic losses (fewer steps $\approx$ larger penalty).

7 Trees and MLPs Have Different Inductive Biases

	Tree boosting	MLP
representation	piecewise functions built from stacked feature thresholds	continuous hidden representation from weighted sums and activations
preprocessing	relatively insensitive to the scale of numerical features	sensitive to scaling, missing values, categorical encoding
data size	strong baseline on small/medium tabular data	benefits from sufficient data
computation	many efficient CPU implementations	suited to minibatches and GPU matrix operations

Why MLPs wobble on small tabular data: many weights and embeddings relative to observations; split rules that a tree learns directly must be built from combinations of many neurons; performance changes a lot with scaling and categorical encoding. Tabular features are often heterogeneous, irregular (thresholds, rotation-variant meaning of columns) and contain uninformative columns — properties that favor trees.

8 Another Route: In-Context Learning

8.1 Refitting versus conditioning

MLP and GBM: fit anew	TabPFN, TabICL: use as context
$\mathcal{D}_{\text{train}} \xrightarrow{\text{fit}} \theta_{\mathcal{D}} \xrightarrow{x_q} \hat{y}_q$	$\Big(\underbrace{\{(x_i, y_i)\}_{i \in C}}_{\text{labeled cases}}, \underbrace{x_q}_{\text{new case}}\Big) \xrightarrow{\text{fixed pre-trained model}} \hat{y}_q$
a new model is learned for each new data set	uses a model pre-trained on many tabular tasks
weights or trees are updated from the training data	predicts usually without re-training weights
estimates can be unstable with small data	

Both use the new data. MLP and GBM change the model with the new data; in-context learning keeps the model fixed and feeds the new data as input.

8.2 TabPFN pre-trains the learning algorithm itself

Algorithm: TabPFN

Pre-training. (1) Generate many synthetic tables from diverse generating structures (prior over structural causal models, MLPs, trees, noise, ...).
(2) Split each table into rows with visible labels and rows with hidden labels.
(3) Train to predict the hidden labels from the visible rows.
(4) Repeat over many tabular tasks to learn Θ.

Prediction on a new data set. $(X_{\text{train}}, y_{\text{train}}, x_q) \xrightarrow{\text{fixed } \Phi_\Theta} \hat{p}(y_q)$: the pre-trained weights stay unchanged; a Transformer reads the structure of the whole new table; the inference procedure learned in pre-training computes the distribution of y_q.

TabPFN does not simply compare training rows with the new row; it runs, in one forward pass, the "procedure for inferring a prediction rule from a table" learned during pre-training.

Theorem: what prior-fitted training approximates

Let tasks $\phi \sim \pi$, and given ϕ draw a data set $\mathcal{D}$ and a query (x_q, y_q). If q_Θ ranges over all conditional distributions, the minimizer of the pre-training loss

$$\mathbb{E}_{\phi, \mathcal{D}, x_q, y_q}\big[-\log q_\Theta(y_q \mid x_q, \mathcal{D})\big]$$

is the **posterior predictive distribution** $p(y_q \mid x_q, \mathcal{D}) = \int p(y_q \mid x_q, \phi)\, p(\phi \mid \mathcal{D})\, d\phi$.

Proof. Conditioning on $(\mathcal{D}, x_q)$, the expected loss equals $\mathbb{E}_{\mathcal{D}, x_q}[\mathrm{KL}(p(\cdot \mid x_q, \mathcal{D}) \,\|\, q_\Theta(\cdot \mid x_q, \mathcal{D}))] + \mathbb{E}[H(p(\cdot \mid x_q, \mathcal{D}))]$. The entropy term does not depend on Θ and the KL term is ≥ 0 with equality iff $q_\Theta = p(\cdot \mid x_q, \mathcal{D})$. The posterior predictive form follows from $p(y_q \mid x_q, \mathcal{D}) = \int p(y_q \mid x_q, \phi) p(\phi \mid x_q, \mathcal{D}) d\phi$ and $p(\phi \mid x_q, \mathcal{D}) = p(\phi \mid \mathcal{D})$. ■

So TabPFN approximates Bayesian prediction under its synthetic prior. It works well when the new table "looks like" a draw from that prior, and its quality depends on the prior's coverage.

8.3 TabICL summarizes rows first, then learns in context

Let the input table be $X = (x_{ij}) \in \mathbb{R}^{n \times m}$ with n rows and m features.

1. **Column-wise interpretation**: the meaning of each cell is understood within its column, $(x_{1j}, \ldots, x_{nj}) \longrightarrow (e_{1j}, \ldots, e_{nj})$. The same number 50 is represented differently in the age column and in the income column.

2. **Row-wise summary**: all cell representations of a row are merged into one vector, $(e_{i1}, \ldots, e_{im}) \longrightarrow h_i \in \mathbb{R}^d$, so every row becomes a fixed-length vector regardless of m.

3. **In-context learning**: the prediction rule of the new task is inferred from the row representations, $\left(\{(h_i, y_i)\}_{i=1}^n, h_q\right) \xrightarrow{\text{pre-trained TabICL}} \hat{y}_q$, applying the inference method learned in pre-training to the rows of the current table.

Compressing rows first makes the in-context step scale to more rows than attending over all cells.

Remark: cautions for in-context models

The context must contain only training rows of the current split (otherwise the leakage of Ch. 1 enters through the context). Performance is limited by the context length (number of rows and features the model accepts), by mismatch between the pre-training prior and the real data, and by compute and memory at prediction time, which grows with context size.

9 Summary

Data condition	Candidates to compare first	What to check
small/medium mixed-type data	regularized regression, CatBoost, LightGBM	is performance stable across splits?
enough data or useful categorical embeddings	tree boosting and a small MLP	were preprocessing and overfitting controlled?
pre-training and ICL usable	add TabPFN/TabICL to the existing baselines	do problem type and data size fit, within the compute budget?

MLP. Converts tabular input, through preprocessing and embeddings, into a continuous hidden representation. Minibatch and Adam are optimization devices; weight decay, dropout and early stopping are generalization devices.

TabPFN and TabICL. Pre-trained models that put labeled cases into the context and predict new query rows. They can be useful candidates for small/medium tabular tasks but are sensitive to context length, the pre-training prior and leakage.